

ASP.NET Core Secret Manager

Keeps production secrets like database connection strings, API and encryption keys out of source control

The main use of Secret Manager is to keep production secrets like database connection strings, API and encryption keys out of source control.

Why you should not store secrets in configuration files

We usually store database connection strings, third party service credentials, API and encryption keys in configuration files like `web.config` in class asp.net and `appSettings.json` in asp.net core.

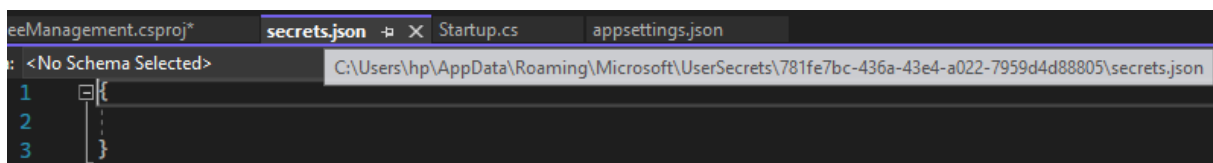
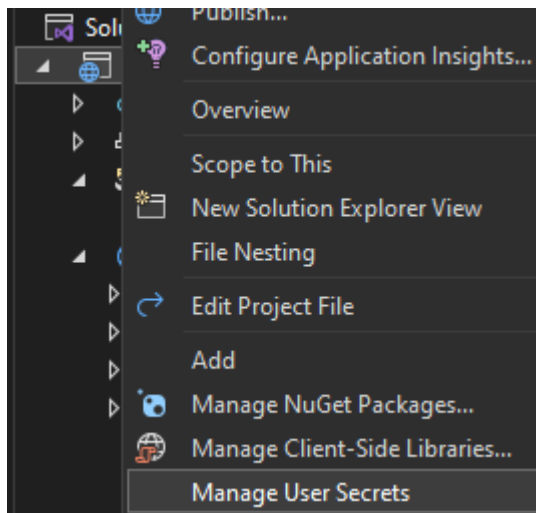
```
https://json.schemastore.org/appsettings.json
{
  "AllowedHosts": "*",
  "MyKey": "Value of MyKey from appsettings.json",
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;database=EmployeeDB;Tr"
  }
}
```

These configuration files are part of the project. So when they are committed to the source control repository, everyone who has access to the repository will have access to the sensitive data in these files and could be misused.

From security standpoint, it is not a good idea, to store passwords or other sensitive data in configuration files or source code.

Secret Manager allows developers to store and retrieve sensitive data during the development of an ASP.NET Core application. It stores sensitive data i.e user secrets in a file with name `secrets.json`.

To add this file to your project, right click on the project name in Solution Explorer in Visual Studio and select `Manage User Secrets` from the context menu. This adds `secrets.json` file.



C:\Users\hp\AppData\Roaming\Microsoft\UserSecrets\781fe7bc-436a-43e4-a022-7959d4d88805

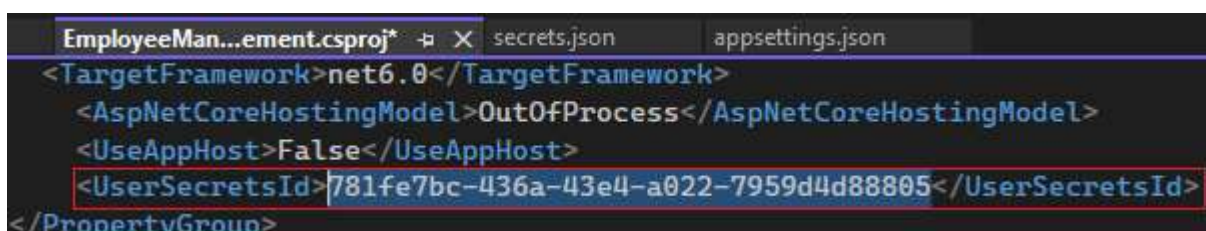
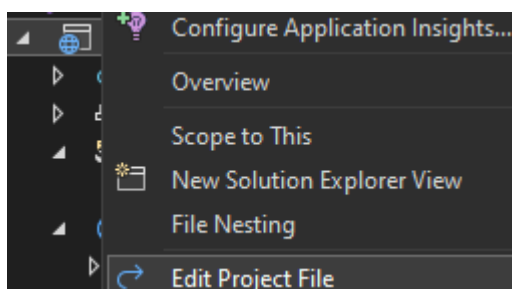
Трябва да се обърне внимание, че файлът не е в папката с проекта. Местоположението му се вижда при посочване с мишката, а структурата му много наподобява appsettings.json.

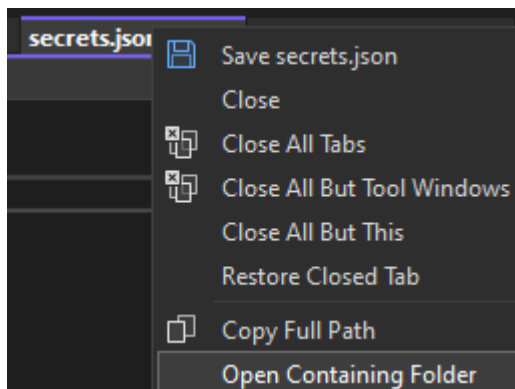
The structure of this file is similar to [appSettings.json](#). The important point to keep in mind is, [this file is not part of the project folder](#). It is located outside of the project folder at the following path.

C:\Users\{UserName}\AppData\Roaming\Microsoft\UserSecrets\{ID}

- **{UserName}** is the windows user name that you use to log into the computer.
- **{ID}** is a GUID (Globally Unique Identifier)

On a single computer you may have multiple asp.net core projects and a **secrets.json** file for each project. It is this GUID, that links a given **secrets.json** file to a given asp.net core project. To establish this link, **UserSecretsId** node is included in the **.csproj** file.





C:\Users\hp\AppData\Roaming\Microsoft\UserSecrets\781fe7bc-436a-43e4-a022-7959d4d88805				
	Name	Date modified	Type	Size
	secrets	7.12.2022 r. 15:49	JSON File	1 KB

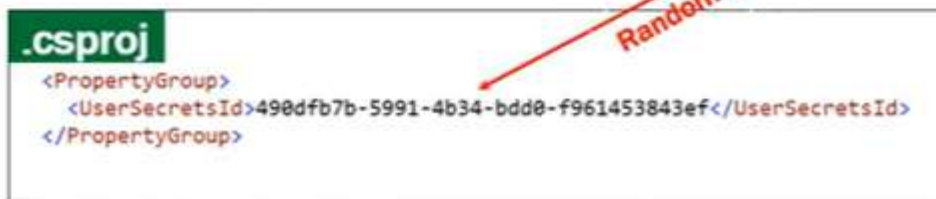
Уникалният номер на UserSecerts съвпада в директорията и файла на проекта.

ASP.NET Core Secret Manager

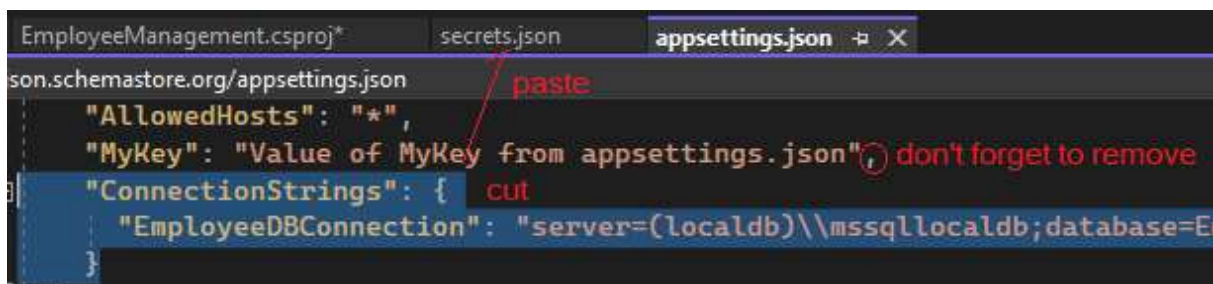
- User secrets are stored in **secrets.json**
- Structure is similar to **appSettings.json**
- Located **outside** of the project folder

C:\Users\[UserName]\AppData\Roaming\Microsoft\UserSecrets\[ID]

↓
Windows Login Name

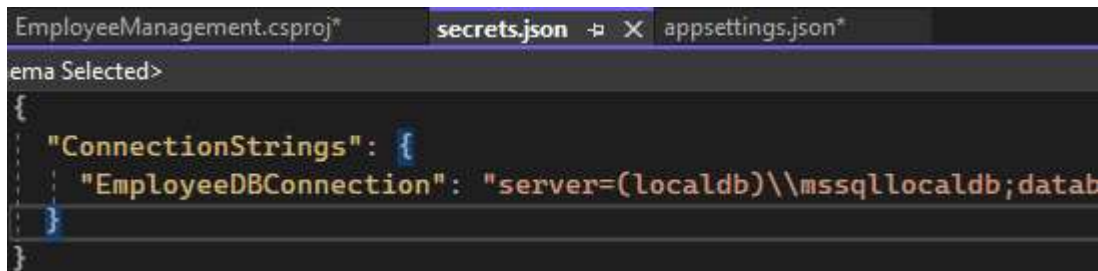


A given **secrets.json** file can be shared by multiple projects.



```
"ConnectionStrings": {
  "EmployeeDBConnection":
"server=(localdb)\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true"
```

}



```
EmployeeManagement.csproj* secrets.json x appsettings.json*
ema Selected>
{
  "ConnectionStrings": {
    "EmployeeDBConnection": "server=(localdb)\\mssqllocaldb;datab
  }
}
```

При старт, приложението работи по същият начин.

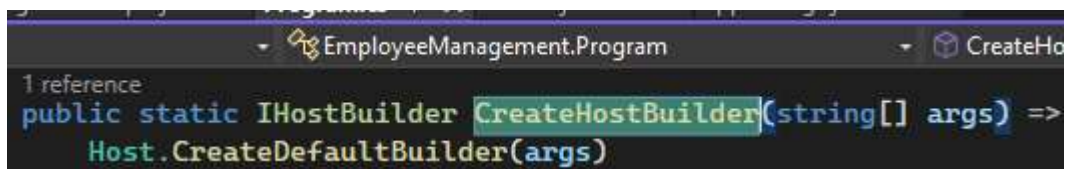
ASP.NET Core Configuration Sources

order to read from IHostBuilder:

- **appsettings.json (Part 9)**
- **User Secrets**
- **Environment variables (Part 14)**
- **Command-line argument**

IConfiguration service is setup to read configuration information from all the various configuration sources

Later configuration sources **override** the earlier configuration sources



```
EmployeeManagement.Program CreateHo
1 reference
public static IHostBuilder CreateHostBuilder(string[] args) =>
    Host.CreateDefaultBuilder(args)
```

<https://github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs>

github.com/aspnet/MetaPackages/blob/release/2.2/src/Microsoft.AspNetCore.WebHost.cs

```

config.AddJsonFile("appsettings.json", optional: true, reloadOnChange: true);
    .AddJsonFile($"appsettings.{env.EnvironmentName}.json", optional: true, reloadOnChange: true);

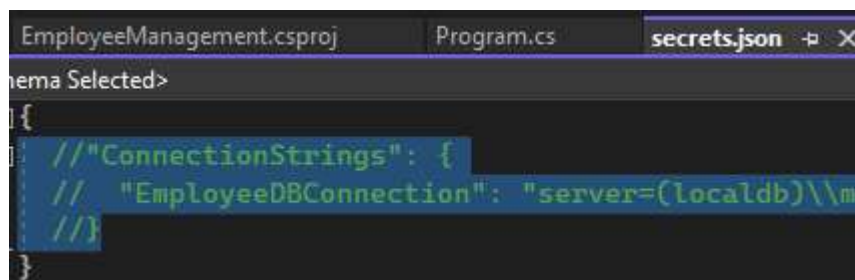
if (env.IsDevelopment())
{
    var appAssembly = Assembly.Load(new AssemblyName(env.ApplicationName));
    if (appAssembly != null)
    {
        config.AddUserSecrets(appAssembly, optional: true);
    }
}

config.AddEnvironmentVariables();

if (args != null)
{
    config.AddCommandLine(args);
}

```

Ако се закоментира препратката към DB – secrets.json

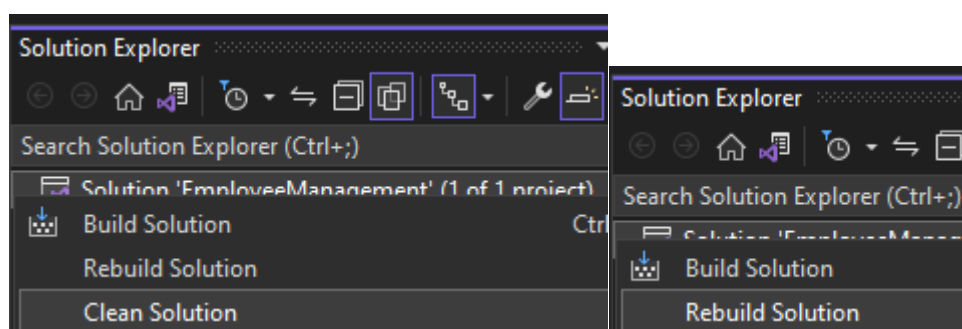


```

{
  //\"ConnectionStrings\": {
  //  \"EmployeeDBConnection\": \"server=(localdb)\\ms
  //}
}

```

тогава никъде няма препратка към нея в целия проект. При старт на проекта, видимо той може да функционира правилно, защото използва заредени стари данни, но ако ги почистим:



Clean Solution,
Rebuild Solution

An unhandled exception occurred while processing the request.

ArgumentException: Value cannot be null. (Parameter 'connectionString')

Microsoft.EntityFrameworkCore.Utilities.Check.NotEmpty(string value, string parameterName)

Stack Query Cookies Headers Routing

ArgumentException: Value cannot be null. (Parameter 'connectionString')

Microsoft.EntityFrameworkCore.Utilities.Check.NotEmpty(string value, string parameterName)

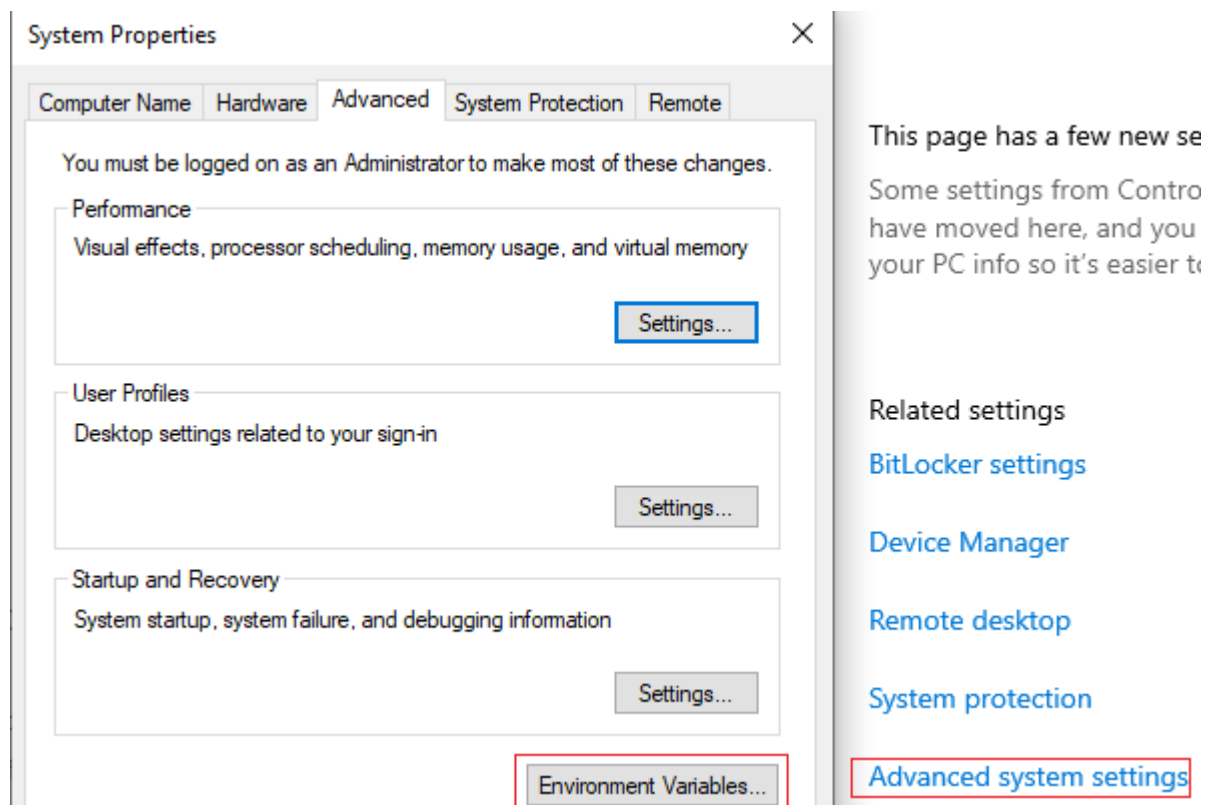
Microsoft.EntityFrameworkCore.SqlServer.DbContextOptionsExtensions.UseSqlServer(DbContextOptionsBuilder optionsBuilder, string connectionString, Action<SqlServerDbContextOptionsBuilder> sqlServerOptionsAction)

EmployeeManagement.Startup.<ConfigureServices>b__2_0(DbContextOptionsBuilder options) in Startup.cs

32. options => options.UseSqlServer(_config.GetConnectionString("EmployeeDBConnection"));

connectionString is null.

Run -> Control Panel -> System -> Advanced -> Environment Variables -> New:

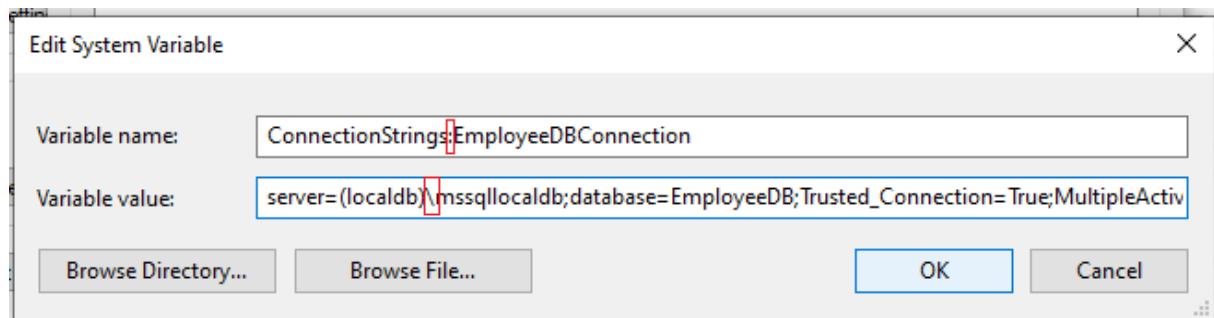


ConnectionStrings:EmployeeDBConnection

```
server=(localdb)\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true
```

```
without 1 \
```

```
server=(localdb)\mssqllocaldb;database=EmployeeDB;Trusted_Connection=True;MultipleActiveResultSets=true
```



Restart VS -> works.

User secrets in production

To protect sensitive data, `secrets.json` file is deliberately kept outside of the project folder. This file is not checked into source control repository. This means `secrets.json` file is not copied onto the production server, when we actually build and deploy. So, where will the application find database connection string.

Well, on a production server store the database connection string in an environment variable. If you remember, `IConfiguration` service is setup to read configuration information from all the following configuration sources.

- appsettings.json
- User secrets
- Environment variables
- Command-line argument

This means, in spite of not having `secrets.json` file on the production server, our application should work just fine because it will find the required database connection string in the environment variable.

Secret Manager isn't for staging or production server, it should only be used on development machine. For production always use either environment variables, Azure Key Vault, or 3rd party production secret management system.